

练习 14.24: 你在 7.5.1 节的练习 7.40 (第 261 页) 中曾经选择并编写了一个类, 你认为它应该含有拷贝赋值和移动赋值运算符吗? 如果是, 请实现它们。

练习 14.25: 上题的这个类还需要定义其他赋值运算符吗? 如果是, 请实现它们; 同时说明运算对象应该是什么类型并解释原因。

14.5 下标运算符

表示容器的类通常可以通过元素在容器中的位置访问元素, 这些类一般会定义下标运算符 `operator[]`。



下标运算符必须是成员函数。

565

为了与下标的原始定义兼容, 下标运算符通常以所访问元素的引用作为返回值, 这样做的好处是下标可以出现在赋值运算符的任意一端。进一步, 我们最好同时定义下标运算符的常量版本和非常量版本, 当作用于一个常量对象时, 下标运算符返回常量引用以确保我们不会给返回的对象赋值。



如果一个类包含下标运算符, 则它通常会定义两个版本: 一个返回普通引用, 另一个是类的常量成员并且返回常量引用。

举个例子, 我们按照如下形式定义 `StrVec` (参见 13.5 节, 第 465 页) 的下标运算符:

```
class StrVec {
public:
    std::string& operator[](std::size_t n)
    { return elements[n]; }
    const std::string& operator[](std::size_t n) const
    { return elements[n]; }
    // 其他成员与 13.5 (第 465 页) 一致
private:
    std::string *elements;           // 指向数组首元素的指针
};
```

上面这两个下标运算符的用法类似于 `vector` 或者数组中的下标。因为下标运算符返回的是元素的引用, 所以当 `StrVec` 是非常量时, 我们可以给元素赋值; 而当我们对常量对象取下标时, 不能为其赋值:

```
// 假设 svec 是一个 StrVec 对象
const StrVec cvec = svec;           // 把 svec 的元素拷贝到 cvec 中
// 如果 svec 中含有元素, 对第一个元素运行 string 的 empty 函数
if (svec.size() && svec[0].empty()) {
    svec[0] = "zero";               // 正确: 下标运算符返回 string 的引用
    cvec[0] = "Zip";                // 错误: 对 cvec 取下标返回的是常量引用
}
```

566

14.5 节练习

练习 14.26: 为你的 StrBlob 类、StrBlobPtr 类、StrVec 类和 String 类定义下标运算符。

14.6 递增和递减运算符

在迭代器类中通常会实现递增运算符(++)和递减运算符(--),这两种运算符使得类可以在元素的序列中前后移动。C++语言并不要求递增和递减运算符必须是类的成员,但是因为它们改变的正好是所操作对象的状态,所以建议将其设定为成员函数。

对于内置类型来说,递增和递减运算符既有前置版本也有后置版本。同样,我们也应该为类定义两个版本的递增和递减运算符。接下来我们首先介绍前置版本,然后实现后置版本。

Best Practices

定义递增和递减运算符的类应该同时定义前置版本和后置版本。这些运算符通常应该被定义成类的成员。

定义前置递增/递减运算符

为了说明递增和递减运算符,我们不妨在 StrBlobPtr 类(参见 12.1.6 节,第 421 页)中定义它们:

```
class StrBlobPtr {
public:
    // 递增和递减运算符
    StrBlobPtr& operator++();           // 前置运算符
    StrBlobPtr& operator--();
    // 其他成员和之前的版本一致
};
```

Best Practices

为了与内置版本保持一致,前置运算符应该返回递增或递减后对象的引用。

567

递增和递减运算符的工作机理非常相似:它们首先调用 check 函数检验 StrBlobPtr 是否有效,如果是,接着检查给定的索引值是否有效。如果 check 函数没有抛出异常,则运算符返回对象的引用。

在递增运算符的例子中,我们把 curr 的当前值传递给 check 函数。如果这个值小于 vector 的大小,则 check 正常返回;否则,如果 curr 已经到达了 vector 的末尾,check 将抛出异常:

```
// 前置版本:返回递增/递减对象的引用
StrBlobPtr& StrBlobPtr::operator++()
{
    // 如果 curr 已经指向了容器的尾后位置,则无法递增它
    check(curr, "increment past end of StrBlobPtr");
    ++curr;           // 将 curr 在当前状态下向前移动一个元素
    return *this;
}
```